# American Computer Science League

2025 Finals ● Short Problems Solutions ● Senior Division

**1. Boolean Algebra**

$\overline{A}(\overline{B} + C) + \overline{AB}(C + \overline{A}) + \overline{ABC}$
$= \overline{A}\,\overline{B} + \overline{A}\,C + (\overline{A} + \overline{B})(C + \overline{A}) + \overline{A} + \overline{B} + \overline{C}$
$= \overline{A}\,\overline{B} + \overline{A}\,C + \overline{A}C + \overline{A}\,\overline{A} + \overline{B}C + \overline{B}\,\overline{A} + \overline{A} + \overline{B} + \overline{C}$
$= \overline{A}(\overline{B} + C + C + 1 + \overline{B} + 1) + \overline{B}(C + 1) + \overline{C}$
$= \overline{A} + \overline{B} + \overline{C}$
$= \overline{ABC}$
This is FALSE for only the ordered triple (1,1,1).
Alternatively, the first 2 terms are not needed because the last term yields everything but (1,1,1), which includes the others.

C. (1,1,1)

**2. Boolean Algebra**

$\overline{A + B}\,(\overline{A} + \overline{BC}) + \overline{A}\,\overline{B}(\overline{\overline{ABC}})$
$= \overline{A}\,\overline{B}\,(\overline{A} + \overline{B} + \overline{C}) + \overline{A}\,\overline{B}(\overline{\overline{AB}} + \overline{C})$
$= \overline{A}\,\overline{B}\,(\overline{A} + \overline{B} + \overline{C}) + \overline{A}\,\overline{B}(AB + \overline{C})$
$= \overline{A}\,\overline{B}\,\overline{A} + \overline{A}\,\overline{B}\,\overline{B} + \overline{A}\,\overline{B}\,\overline{C} + \overline{A}\,\overline{B}AB + \overline{A}\,\overline{B}\,\overline{C}$
$= \overline{A}\,\overline{B} + \overline{A}\,\overline{B} + \overline{A}\,\overline{B}\,\overline{C} + 0 + \overline{A}\,\overline{B}\,\overline{C}$
$= \overline{A}\,\overline{B} + \overline{A}\,\overline{B}\,\overline{C}$
$= \overline{A}\,\overline{B}(1 + \overline{C})$
$= \overline{A}\,\overline{B}$
$= \overline{A + B}$ which is a NOR gate

C. NOR

**3. Digital Electronics**

The Boolean expression that is represented by the circuit is:
$\overline{AB + \overline{(B + C)C}}$

$\overline{AB + \overline{(B + C)C}}$
$= \overline{AB + \overline{B + C} + \overline{C}}$
$= \overline{AB + \overline{B}\,\overline{C} + \overline{C}}$
$= \overline{AB + \overline{C}(\overline{B} + 1)}$
$= \overline{AB + \overline{C}}$
$= \overline{AB}\,C$
$= (\overline{A} + \overline{B})C$
To be TRUE, $(\overline{A} + \overline{B}) = 1\ and\ C = 1.$
Therefore (0,1,1), (1,0,1), and (0,0,1) make it TRUE.

B. 3

**4. Bit-String Flicking**

(LSHIFT-1 (RCIRC-2 (RSHIFT-3 (LCIRC-1 (NOT 10110)))))
= (LSHIFT-1 (RCIRC-2 (RSHIFT-3 (LCIRC-1 01001))))
= (LSHIFT-1 (RCIRC-2 (RSHIFT-3 10010)))
= (LSHIFT-1 (RCIRC-2 00010))
= (LSHIFT-1 10000)
= 00000

A. 00000

**5. Bit-String Flicking**

(LCIRC-2 X) OR (RSHIFT-1 (RCIRC-2 (X AND 01110))) =
(LSHIFT-2 (11010 OR X))

Let X = abcde
LHS = (LCIRC-2 X) OR (RSHIFT-1 (RCIRC-2 (X AND 01110)))
= (LCIRC-2 abcde) OR (RSHIFT-1 (RCIRC-2 (abcde AND 01110)))
= cdeab OR (RSHIFT-1 (RCIRC-2 0bcd0))
= cdeab OR (RSHIFT-1 d00bc)
= cdeab OR 0d00b
= cdeab
RHS = (LSHIFT-2 (11010 OR X))
= (LSHIFT-2 11c1e)
= c1e00
LHS = RHS → cdeab = c1e00
Therefore, a=0, b=0, c=*, d=1, e=*

D. 00*1*

**6. Recursive Functions**

$f(10,10) = 2 * f(10 - 3,10 + 1) - 3 = 2 * f(7,11) - 3 = 2 * 21 - 3 = 39$
$f(7,11) = 2 * f(7 - 3,11 + 1) - 3 = 2 * f(4,12) - 3 = 2 * 12 - 3 = 21$
$f(4,12) = f(4 + 1,12 - 4) + 1 = f(5,8) + 1 = 11 + 1 = 12$
$f(5,8) = f(5 + 1,8 - 4) + 1 = f(6,4) + 1 = 10 + 1 = 11$
$f(6,4) = 6 + 4 = 10$

D. 39

**7. Recursive Functions**

$f(50) = f(25 - 1) + 3 = f(24) + 3 = 4 + 3 = 7$
$f(24) = f(12 - 1) + 3 = f(11) + 3 = 1 + 3 = 4$
$f(11) = f(11 - 4) - 1 = f(7) - 1 = 2 - 1 = 1$
$f(7) = [7/3] = 2$
So $f(f(f(50))) = f(f(7)) = f(2) = 5$
$f(2) = f(1 - 1) + 3 = f(0) + 3 = 2 + 3 = 5$
$f(0) = f(0 - 1) + 3 = f(-1) + 3 = -1 + 3 = 2$
$f(-1) = [-⅓] = -1$

D. 5

**8. Prefix/Infix/Postfix Notation**

/ * ! * 2 5 % * 9 + 9 2 * ! 8 % * ^ 2 3 5
= / * ! (* 2 5) % * 9 (+ 9 2) * (! 8) % * (^ 2 3) 5
= / * ! 10 % * 9 11 * (! 8) % * 8 5
= / * (! 10) % (* 9 11) * (! 8) % (* 8 5)
= / * (10!) % 99 * (8!) % 40
= / * (10!) (% 99) * (8!) (% 40)
= / * (10!) 9 * (8!) 6
= / (* (10!) 9) (* (8!) 6)
= ( 10! * 9) / (8! * 6)
= (10 *9 * 8! * 9) / (8! * 6)
= 135

C. 135

**9. Prefix/Infix/Postfix Notation**

2 3 + 5 2 5 3 * + * − 8 4 / −
= (((2 3 +) (5 (2 (5 3 *) +) *) −) (8 4 /) −)
= (((+ 2 3) (5 (2 (* 5 3) +) *) −) (/ 8 4) −)
= (((+ 2 3) (* 5 (+ 2 (* 5 3))) −) (/ 8 4) −)
= ((− (+ 2 3) (* 5 (+ 2 (* 5 3)))) (/ 8 4) −)
= (− (− (+ 2 3) (* 5 (+ 2 (* 5 3)))) (/ 8 4))
= − − + 2 3 * 5 + 2 * 5 3 / 8 4

A. − − + 2 3 * 5 + 2 * 5 3 / 8 4

**10. Computer Number Systems**

$475_8 + 567_8 = 1264_8$
$1010110100_2 * 100_2 = 101011010000_2 = AD0_{16}$

D. AD0

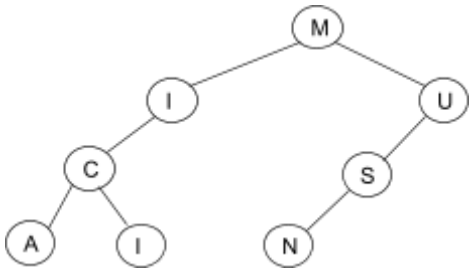
**11. Computer Number Systems**

$62_8$ meals served = 50 meals served
$11110_2$ salmon = 30 salmon, $14_{16}$ beef = 20 beef
Price: salmon $\$12_{16} = \$18$, beef $\$30_8 = \$24$
Cost: salmon 30 @ \$18 = \$540, beef 20 @ \$24 = \$480
Total food bill: \$540 + \$480 = \$1020 = $3FC_{16}$

C. 3FC

**12. Data Structures**

Since the preorder traversal is **MICAIUSN** and the inorder traversal is **ACIIMNSU**, M is the root, I goes to the left, C goes to the left, and A goes to the left. I is on the right of C, U is on the right of M, S is on the left of U and N is on the left of S. Therefore, the postorder traversal is AICINSUM. The tree looks like:




D. AICINSUM

**13. Data Structures**

This algorithm will accept any valid postfix expression. For the correct answer, a b c d e f − + * / ^, push each letter. Then for each operator, pop two items and push a # symbol. For all but C, there are too many, too few, or incorrectly placed operators.

A. a * b c d e / + / f −
a → tries to pop from an empty stack
B. a b + c d / e / f −
a → ab → # → #c → #cd → ## → ##e → ## → ##f → ##
C. a b c d e f − + * / ^
a → ab → abc → abcd → abcde → abcdef → abcd# → abc# → ab# → a# → #
D. a b * c d / e f / * + ^
a → ab → # → #c → #cd → ## → ##e → ##ef → ### → ## → # → tries to pop from an empty stack

C. a b c d e f − + * / ^

**14. Graph Theory**

The 3 ingredient meals are:

beef, rice, corn
beef, potato, corn
beef, potato, yam

rice, corn, shrimp

shrimp, yam, beef
shrimp, beef, rice
shrimp, beef, potato

yam, beef, rice

corn, shrimp, beef
corn, shrimp, yam

potato, corn, shrimp

There are 11 different meals.

B. 11

**15. Graph Theory**

By inspection, all of the cycles from F not containing A or C are: FBF, FGBF, FEF, and FGF.

B. 4

**16. What Does This Program Do?**

```
num = 2025
x = 1
while num != 0
   x = x + 1
   if num % x == 0 then
      n = num
      t = 0
      while n % x == 0
         t = t + 1
         n = n / x
      end while
      output x,t
      num = num / x ^ t
   end if
end while
```

The main while loop continues as long as there are more prime factors to be found. It checks sequential values of x starting with 2 to see if it goes in evenly. If it does, the program uses the second while loop to find how many times it goes into the current value of num. The prime factorization of 2025 is 3^4*5^2 so the output is 3 4 on one line and 5 2 on the next.

A. 3 4
5 2

**17. LISP**

For the list (a (b c) (d (e f))), ((e f)) is the 3rd item so two CDRs are needed first and then a CAR is needed to get the first item in the resulting list: (d (e f)). The CDR of that is the list containing the second item which is ((e f)).

A. (CDDDR x) = (CDDDR '(a (b c) (d (e f))))
= (CDDR '((b c) (d (e f))))
= (CDR '((d (e f))))
= ()

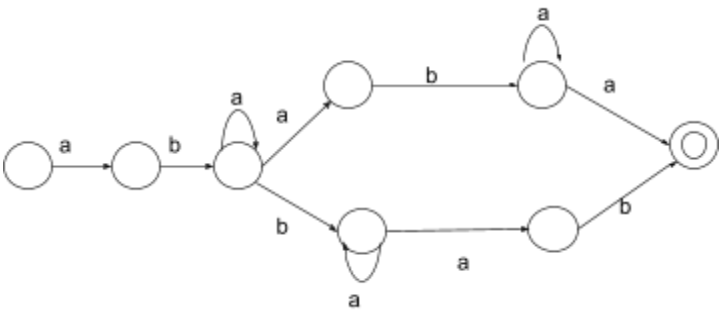
B. (CDDADR x) = (CDDADR '(a (b c) (d (e f))))
= (CDDAR '((b c) (d (e f))))
= (CDDR '(b c))
= (CDR '(c))
= ()

C. (CDADDR x) = (CDADDR '(a (b c) (d (e f))))
= (CDADR '((b c) (d (e f))))
= (CDAR '((d (e f))))
= (CDR '(d (e f)))
= ((e f))

D. (CADDR x) = (CADDR '(a (b c) (d (e f))))
= (CADR '((b c) (d (e f))))
= (CAR '((d (e f))))
= (d (e f))

C. (CDADDR x)

**18. FSAs and Regular Expressions**




Beginning: aba*
Top branch: aba*a
Lower branch: ba*ab
Combined: aba* (aba*a ∪ ba*ab)

C. aba* (aba*a ∪ ba*ab)

**19. FSAs and Regular Expressions**

The regular expression is:

```
[abc][aeiou]*[lx]*[^g]*[o-x]*
```

a) afghan - fails at g in [o-x]*
b) bulldog - fails at g in [^g]*
c) collie - satisfies the regular expression
d) boxer - satisfies the regular expression
e) corgi - fails at g in [[o-x]*
f) akita - satisfies the regular expression
g) husky - fails at h in [abc]
h) poodle- fails at p in [abc]

C. c, d, f

**20. Assembly Language Programming**

```
T     DC      0
N     DC      2025
      LOAD    N
      STORE   NUM
TOP   DIV     =10
      STORE   Q
      MULT    =10
      SUB     N
      MULT    =-1
      ADD     T
      STORE   T
      LOAD    Q
      BE      STOP
      STORE   N
      BU      TOP
STOP  LOAD    NUM
      DIV     T
      STORE   ANS
      PRINT   ANS
      END
```

The number 2025 is a Harshad number because it is divisible by the sum of its digits. This program finds the sum of the digits inside the loop by using integer division to get the remainder of dividing by 10. After the loop, it divides the original number 2025 by the sum that is stored in T which is 9. Therefore, 2025/9 = 225.

B. 225